ASSIGNMENT 1 – CO1

Assignment Title

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Name: Lakshmi Devi J

Register Number: 192524300

GitHub Repository Link: https://github.com/laxmijaya0812-prog/DSA0119--C/tree/main

TASK 1 – Understand the Problem and Identify the Objects

Aim

To analyse the Metro Smart-Card Fare System using Object-Oriented Programming concepts and identify its objects, attributes, behaviours and rules.

1. Problem Description

The Namma Metro system is divided into three zones:

- Inner Zone – Rs.10
- Middle Zone – Rs.20
- Outer Zone – Rs.30

A 20% surcharge is applied during peak hours:

- 8:00 AM to before 10:00 AM
- 5:00 PM to before 8:00 PM

A journey is allowed only when at least Rs.50 remains in the smart card after deducting the fare.

Students and senior citizens are also eligible for concessions.

The system must calculate fares accurately and prevent negative balances.

2. Identify the Objects

Object	Data / Attributes	Behaviour / Methods
SmartCard	Card ID, balance	Check balance, deduct fare, recharge
Passenger	Passenger ID, name, category	Identify passenger type, apply concession
Journey	Source, destination, time	Calculate journey fare
Zone	Zone name, zone fare	Identify travelled zone
FareTable	Base fares, surcharge	Calculate final fare

SmartCard

The SmartCard object stores the passenger's balance. Its balance should be private so that outside functions cannot directly change it.

Passenger

The Passenger object stores passenger information and category. It can be used to apply student or senior-citizen concessions.

Journey

The Journey object stores journey details such as source, destination and travel time. It determines the farthest zone and fare.

Zone

The Zone represents Inner, Middle and Outer zones and their corresponding fares.

FareTable

The FareTable stores the standard fares and surcharge rules in one place. This avoids writing fare values repeatedly.

3. Procedural Programming vs Object-Oriented Programming

Procedural Approach

A procedural implementation would use global variables and separate functions such as:

calculateFare();

checkBalance();

deductFare();

The data and functions are separate.

Object-Oriented Approach

An OOP implementation groups data and related functions inside classes.

For example:

SmartCard card;

Journey journey;

FareTable table;

The OOP approach provides better encapsulation, reuse and maintenance.

4. Basic Principles of OOP

Encapsulation

Data and functions are combined inside a class.

Example:

```
class SmartCard {

private:

    long long balance;

};
```

The balance cannot be directly modified from outside the class.

Abstraction

Unnecessary implementation details are hidden from the user.

For example, the passenger only uses the card to make a journey without knowing the internal fare calculation.

Inheritance

Different passenger types can inherit common properties from a Passenger class.

Example:

class Student : public Passenger {};

class SeniorCitizen : public Passenger {};

Polymorphism

Different passenger categories can implement different concession rules using the same interface.

For example:

calculateConcession();

can behave differently for students and senior citizens.

5. Rules Every Correct Journey Must Satisfy

1. The zone must be valid.
2. The fare must be non-negative.
3. Peak surcharge must be applied only during valid peak hours.
4. The balance must not become negative.
5. At least Rs.50 must remain after fare deduction.
6. The fare must be calculated using exact integer paise.
7. Invalid journeys must be rejected.

Alternative Concept and Complexity Analysis

If the system were implemented using global variables and free functions, the fare calculation and balance checking would be handled separately.

Example:

long long balance = 10000;

```
bool canTravel(long long fare) {
    return balance - fare >= 5000;
}
void deductFare(long long fare) {
    if (canTravel(fare))
        balance -= fare;
}
```

Both procedural and OOP approaches require **O(1)** time for one journey and **O(1)** memory per card.

However, OOP is easier to maintain because changes to the balance rule can be made inside the SmartCard class instead of modifying several global functions.

Result

Thus, the Metro Smart-Card System was analysed using OOP principles. SmartCard, Passenger, Journey, Zone and FareTable were identified as important objects. OOP is preferred because it provides encapsulation, abstraction, reusability and easier maintenance.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly

Aim

To identify C++ tokens and define metro fares, surcharge and limits using appropriate constants and enumerations.

1. Five Types of Tokens

Keywords

Keywords have predefined meanings in C++.

Examples:

class

int

return

if

const

Identifiers

Identifiers are names given to variables, functions and classes.

Examples:

balance

fare

SmartCard

calculateFare

Constants

Constants are fixed values that should not be changed.

Examples:

1000

2000

3000

20

5000

Operators

Operators perform operations.

Examples:

+

-

*

/

%

&&

||

=

Punctuators

Examples:

;

{}

()

[]

,

2. #define, const and constexpr

#define

#define INNER_FARE 1000

It is a preprocessor macro and does not provide normal C++ type checking.

const

const int INNER_FARE = 1000;

The value cannot be modified after initialization.

constexpr

constexpr long long INNER_FARE = 1000;

The value is known at compile time.

constexpr is preferred because it provides type safety, scope and compile-time evaluation.

3. enum and enum class

```
enum class Zone {
    Inner,
    Middle,
    Outer
};
```

enum class is safer because its values are scoped and do not automatically convert to integers.

4. Recommended Constants

```
constexpr long long INNER_FARE_PAISE = 1000;
constexpr long long MIDDLE_FARE_PAISE = 2000;
constexpr long long OUTER_FARE_PAISE = 3000;

constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
```

5. Naming Conventions

- Classes → PascalCase
- Variables → camelCase
- Constants → UPPER_SNAKE_CASE
- Functions → descriptive names

Examples:

```
class SmartCard;
long long balance;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
calculateFare();
```

Alternative Concept and Complexity Analysis

If #define macros were used instead of constexpr, the program would still work, but macros do not provide normal C++ type checking or scope.

Feature	**#define**	**constexpr**
Type checking	Limited	Yes
Scope	Preprocessor scope	C++ scope
Compile-time value	Yes	Yes
Memory	Usually no storage	Usually no storage
Safety	Lower	Higher

The running time is **O(1)** and the additional memory is effectively **O(1)** in both approaches.

Result

Thus, constexpr and enum class were selected because they provide safer, clearer and maintainable definitions for the Metro Smart-Card System.

TASK 3 – Choose Data Types and Check Type Compatibility

Aim

To select suitable C++ data types for the Metro Smart-Card System and understand type compatibility, memory size and precision.

1. Data Types Used

Data	**Data Type**	**Reason**
Card ID	int	Stores card number
Balance	long long	Stores money in paise
Fare	long long	Accurate fare storage

Passenger name	string	Stores text
Peak hour	bool	True/false value
Zone number	int	Stores zone value
Journey count	long long	Supports large counts
Hour	int	Stores hour from 0–23
Concession	double	Represents percentage if required

2. Why Money Should Be Stored in Integer Paise

Money should be stored as integer paise instead of double rupees.

For example:

Rs.10 = 1000 paise

Rs.20 = 2000 paise

Rs.30 = 3000 paise

Advantages:

1. Avoids floating-point rounding errors.
2. Provides exact fare calculations.
3. Prevents paisa-level differences.
4. Makes balance calculations reliable.

3. C++ Program

```
#include <iostream>

#include <climits>

using namespace std;

int main()
```

```
{
    int cardID = 101;
    long long balance = 10000;
    long long fare = 2000;
    bool peak = true;
    double concession = 10.5;
    cout << "Card ID: " << cardID << endl;
    cout << "Balance: " << balance << " paise" << endl;
    cout << "Fare: " << fare << " paise" << endl;
    cout << "Peak Hour: " << peak << endl;
    cout << "Concession: " << concession << "%" << endl;
    cout << "Size of int: " << sizeof(int) << endl;
    cout << "Size of long long: " << sizeof(long long) << endl;
    cout << "Size of double: " << sizeof(double) << endl;
    cout << "Maximum int: " << INT_MAX << endl;
    return 0;
}
```

Output

```
Card ID: 101
Balance: 10000 paise
Fare: 2000 paise
Peak Hour: 1
Concession: 10.5%
Size of int: 4
Size of long long: 8
Size of double: 8
Maximum int: 2147483647
```

4. Type Compatibility

Type compatibility means that C++ can convert one type into another during an expression.

Example:

int a = 10;

double b = 5.5;

double c = a + b;

Here, a is converted from int to double.

The result is:

15.5

5. Why int May Not Be Enough

For 3,000,000 journeys at Rs.30:

3,000,000 × 3000 paise

= 9,000,000,000 paise

The maximum value of a 32-bit signed int is:

2,147,483,647

Therefore, int is not sufficient.

long long should be used for network-level totals.

Alternative Concept and Complexity Analysis

If money were stored as double rupees, addition would still take **O(1)** time and a double generally occupies 8 bytes.

However, floating-point values may introduce small rounding errors.

Integer paise provides exact monetary calculations.

Feature	Integer Paise	Double Rupees
Addition	O(1)	O(1)
Storage	8 bytes for long long	8 bytes
Accuracy	Exact for paise	Possible rounding
Suitable for money	Yes	Less suitable

Result

Thus, integer paise with long long is recommended for monetary values because it provides exact calculations and prevents overflow for large network totals.

TASK 4 – Control Implicit Conversions

Aim

To understand implicit type conversions in C++ and use explicit conversions to improve the reliability of the fare system.

1. Implicit Conversion

An implicit conversion occurs automatically when C++ converts one data type into another.

Examples:

int a = 7;

double b = 2.0;

double result = a / b;

The integer is converted to double.

2. C++ Program

#include <iostream>

```
using namespace std;

int main()

{

    int a = 7, b = 2;

    cout << a / b << endl;

    cout << static_cast<double>(a) / b << endl;

    char ch = 'A';

    cout << ch + 2 << endl;

    cout << static_cast<char>(ch + 2) << endl;

    bool flag = 5;

    cout << flag << ' ' << boolalpha << flag << '\n';


    unsigned int u = 1;

    int s = -1;

    cout << (s < u ? "s < u" : "s >= u") << '\n';

    return 0;

}
```

Output

```
3
3.5
67
C
1 true
s >= u
```

3. Predicted Output

3

3.5

67

C

1 true

s >= u

4. Explanation

a / b

Both values are integers.

7 / 2 = 3

Integer division removes the fractional part.

static_cast<double>(a) / b

a is converted to double.

7.0 / 2 = 3.5

'A' + 2

The character 'A' is promoted to its integer character code.

65 + 2 = 67

static_cast<char>(ch + 2)

67 represents 'C'.

bool flag = 5

Any non-zero integer converts to true.

Signed and unsigned comparison

The signed value -1 is converted to unsigned before comparison. Therefore the comparison does not behave like an ordinary signed comparison.

5. Corrected Code Using Explicit Conversion

```
#include <iostream>

using namespace std;

int main()

{

   int a{7}, b{2};

   double c = static_cast<double>(a) / b;

   cout << c << endl;

   int value = static_cast<int>('A') + 2;

   char ch = static_cast<char>(value);

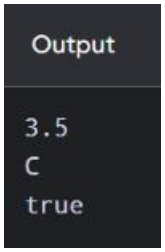
   cout << ch << endl;

   bool flag = (5 != 0);

   cout << boolalpha << flag << endl;

   return 0;

}
```



6. Alternative Concept and Complexity Analysis

Explicit conversions using static_cast make the programmer's intention clear.

Feature	Implicit	Explicit
Conversion	Automatic	Programmer controlled
Readability	Lower	Higher
Type safety	Less control	Better control
Time	O(1)	O(1)
Space	O(1)	O(1)

Result

Thus, implicit and explicit conversions were studied. Explicit conversions using static_cast improve readability and reduce unexpected conversion problems.

TASK 5 – Write Correct Expressions with the Right Precedence

Aim

To understand operator precedence and associativity and write correct expressions for peak hours, surcharge and concessions.

1. Peak-Hour Check

(hour >= 8 && hour < 10) ||

(hour >= 17 && hour < 20)

This represents:

- 8:00 AM to before 10:00 AM
- 5:00 PM to before 8:00 PM

The && operations are evaluated before ||.

2. Surcharge Calculation

fare = fare + (fare * 20 / 100);

The multiplication and division are performed before addition.

For Rs.30:

30 × 20 / 100 = 6

30 + 6 = 36

Therefore, the peak fare is Rs.36.

3. Concession

A conditional operator can be used:

fare = student ? fare * 90 / 100 : fare;

If the passenger is a student, a 10% concession is applied.

4. x++ and ++x

Post-increment

x++

Uses the current value first and then increments it.

Pre-increment

++x

Increments the value first and then uses the new value.

i = i++ should not be used because it creates problematic/undefined behaviour in older C++ rules and is not a safe programming practice.

5. Division and Modulus with Negative Values

10 / 3 = 3

10 % 3 = 1

-10 / 3 = -3

-10 % 3 = -1

6. Predict the Output

Given:

```
#include <iostream>

int main()

{

   int x = 5, y = 3;

   std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

   int z = x++ + ++y;

   std::cout << z << ' ' << x << ' ' << y << '\n';

   std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';

   std::cout << 10 / 3 << ' ' << 10 % 3 << ' '

             << -10 / 3 << ' ' << -10 % 3 << '\n';

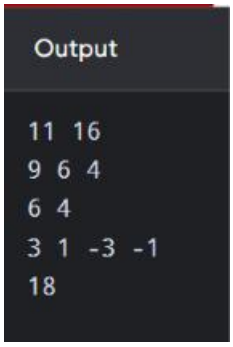
   int k = 0;

   for (int i = 0; i < 10; i += 3)

      k += i;

   std::cout << k << '\n';

}
```



My Predicted Output

11 16

9 6 4

6 4

3 1 -3 -1

18

7. Explanation

Line 1

x + y * 2

Multiplication occurs first:

3 × 2 = 6

5 + 6 = 11

(x + y) * 2

Parentheses are evaluated first:

(5 + 3) × 2 = 16

Line 2

x++ + ++y

x++ uses 5 and then changes x to 6.

++y changes y from 3 to 4.

Therefore:

z = 5 + 4 = 9

Line 3

x > y ? x : y

Since:

6 > 4

the result is 6.

x % 4 * 2

Modulo and multiplication have the same precedence and are evaluated left-to-right:

6 % 4 = 2

2 × 2 = 4

Line 4

10 / 3 = 3

10 % 3 = 1

-10 / 3 = -3

-10 % 3 = -1

Line 5

Loop values are:

0, 3, 6, 9

Therefore:

0 + 3 + 6 + 9 = 18

8. Peak-Hour Boundary Testing

Time	Result
8:00 AM	Peak
9:59 AM	Peak
10:00 AM	Off-Peak
5:00 PM	Peak
7:59 PM	Peak
8:00 PM	Off-Peak

The correct expression is:

(hour >= 8 && hour < 10) ||

(hour >= 17 && hour < 20)

Alternative Concept and Complexity Analysis

A lookup table can be used:

```
bool peak[24] = {};
for (int i = 8; i < 10; i++)
    peak[i] = true;
for (int i = 17; i < 20; i++)
    peak[i] = true;
```

Then:

```
if (hour >= 0 && hour < 24 && peak[hour])
    cout << "Peak Hour";
```

Feature	Logical Expression	Lookup Table
Time	O(1)	O(1)
Space	O(1)	O(1)

Comparisons	Multiple	Fewer
Maintenance	Conditions	Table

The lookup table uses 24 Boolean entries but can make changes to peak-hour schedules easier.

Result

Thus, operator precedence, associativity, increment operators and arithmetic operations were studied. Correct expressions ensure accurate peak-hour fare calculations.

TASK 6 – Design the Control Structures

Aim

To design decision-making and looping structures for calculating fares and processing Metro journeys.

1. Use of Control Structures

if–else

Used when conditions such as peak hour and balance need to be checked.

```
if (peak)
    fare += fare * 20 / 100;
else
    fare = fare;
```

switch

A switch is suitable for selecting the base fare according to the zone.

```
switch (z)
{
    case Zone::Inner:
        fare = 1000;
        break;
```

```
    case Zone::Middle:
        fare = 2000;
        break;

    case Zone::Outer:
        fare = 3000;
        break;
}
```

Conditional Operator

Suitable for simple two-way decisions.

```
result = peak ? surchargeFare : baseFare;
```

2. Loop Selection

for loop

Useful when the number of journeys is known.

```
for (int i = 0; i < n; i++)
```

Range-Based for loop

Suitable when processing a collection.

```
for (const Trip& t : trips)
```

while loop

Suitable when the number of iterations is not known in advance.

For the provided trip collection, the **range-based for loop** is the clearest choice.

3. break and continue

continue can skip an invalid journey.

if (invalid)

    continue;

break can terminate processing when a critical condition occurs.

4. What Happens When No Switch Case Matches?

If no case matches and there is no default, none of the case statements execute.

A default case can be used to handle invalid zones.

5. Pseudocode

Algorithm: Calculate Metro Fare

BEGIN

INPUT zone

INPUT journey hour

INPUT balance

IF zone is invalid THEN

    DISPLAY "Invalid Zone"

    STOP

END IF

IF hour is between 8 and before 10

   OR hour is between 17 and before 20

THEN

    peak ← TRUE

ELSE

```
    peak ← FALSE
END IF

SELECT base fare using zone
IF peak = TRUE THEN
    surcharge ← base fare × 20 / 100
    fare ← base fare + surcharge
ELSE
    fare ← base fare
END IF
IF balance - fare >= 5000 THEN
    balance ← balance - fare
    DISPLAY fare and remaining balance
ELSE
    DISPLAY "Denied"
END IF

END
```

6. Complete the Code

```
#include <iomanip>
#include <iostream>
enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
```

```
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
long long fare(Zone z, bool peak)
{
    long long baseFare = 0;
    switch (z)
    {
        case Zone::Inner:
            baseFare = BASE_PAISE[0];
            break;
        case Zone::Middle:
            baseFare = BASE_PAISE[1];
            break;
        case Zone::Outer:
            baseFare = BASE_PAISE[2];
            break;
        default:
            baseFare = 0;
    }

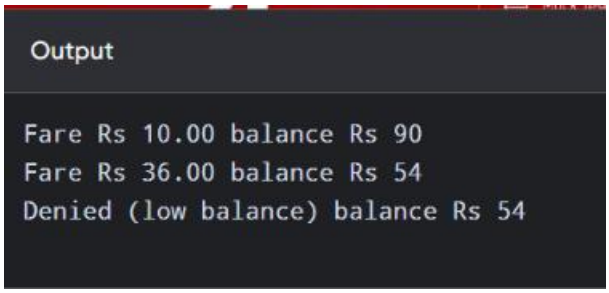
    if (peak)
        baseFare += baseFare * PEAK_SURCHARGE_PERCENT / 100;
    return baseFare;
}
```

```
bool canTravel(long long balance, long long f)
{
    return balance - f >= MIN_BALANCE_PAISE;
}
int main()
{
    long long balance = 10000;
    struct Trip
    {
        Zone zone;
        bool peak;
    };

    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}
    };

    for (const Trip& t : trips)
    {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f))
```

```
        {
            balance -= f;
            std::cout << "Fare Rs "
                      << f / 100 << '.'
                      << std::setw(2)
                      << std::setfill('0')
                      << f % 100
                      << std::setfill(' ');
        }
        else
        {
            std::cout << "Denied (low balance)";
        }
        std::cout << " balance Rs "
                  << balance / 100 << '\n';
    }
}
```



7. Expected Output

Fare Rs 10.00 balance Rs 90

Fare Rs 36.00 balance Rs 54

Denied (low balance) balance Rs 54

8. Boundary Testing

Add journeys at:

10:00 AM

8:00 PM

Both must be treated as **off-peak** because the upper limits use < 10 and < 20.

Alternative Concept and Complexity Analysis

Instead of a switch, an array can be used:

```
constexpr long long FARE[] = {
    1000, 2000, 3000
};
```

Then:

```
long long amount = FARE[zone];
```

Feature	**Switch**	**Array**
Time	O(1)	O(1)
Additional Space	O(1)	O(1)
Readability	Good	Simple
Adding zone	Add case	Add element

The array approach is easier to extend when more zones are added.

Result

Thus, if–else, switch, conditional operators and loops were successfully applied to the Metro fare calculation. Integer paise ensures exact fare calculation.

TASK 7 – From Procedural to Object-Oriented

Aim

To implement the Metro fare system using procedural and object-oriented programming and compare both approaches.

1. Procedural Programming

```
#include <iostream>

using namespace std;

long long balance = 10000;

bool canTravel(long long fare)

{

   return balance - fare >= 5000;

}

void deductFare(long long fare)

{

   if (canTravel(fare))

   {

      balance -= fare;

      cout << "Journey Allowed" << endl;

   }

   else

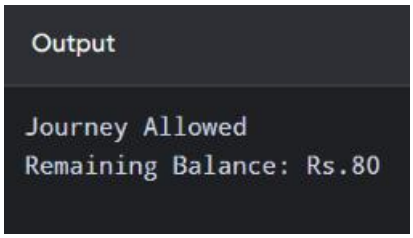
   {

      cout << "Journey Denied" << endl;
```

```
    }
}

int main()
{
    deductFare(2000);
    cout << "Remaining Balance: Rs."
         << balance / 100;

    return 0;
}
```



Output

Journey Allowed

Remaining Balance: Rs.80

2. Object-Oriented Programming

```
#include <iostream>

using namespace std;

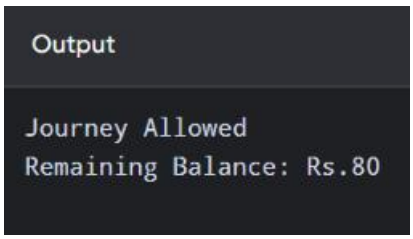
class SmartCard
{
```

```
private:
    long long balance;
public:
    SmartCard(long long b)
    {
        balance = b;
    }

    bool canTravel(long long fare)
    {
        return fare >= 0 &&
               balance - fare >= 5000;
    }

    void deduct(long long fare)
    {
        if (canTravel(fare))
        {
            balance -= fare;
            cout << "Journey Allowed" << endl;
        }
        else
        {
            cout << "Journey Denied" << endl;
```

```
        }
    }
    void display()
    {
        cout << "Remaining Balance: Rs."
             << balance / 100;
    }
};
int main()
{
    SmartCard card(10000);
    card.deduct(2000);
    card.display();
    return 0;
}
```



Output

Journey Allowed

Remaining Balance: Rs.80

3. How SmartCard Prevents Negative Balance

1. The balance is declared private.
2. External functions cannot directly modify it.
3. canTravel() checks whether sufficient balance remains.
4. deduct() changes the balance only after successful validation.
5. The minimum balance of Rs.50 is maintained.

4. Procedural vs OOP

Feature	**Procedural**	**OOP**
Structure	Functions	Classes and objects
Data security	Lower	Higher
Reusability	Limited	High
Maintenance	Difficult	Easier
Data access	Global/separate	Encapsulated
Flexibility	Lower	Higher

Alternative Concept – Public Data Member

If balance were public:

```
class SmartCard
{
public:
   long long balance = 10000;
};
```

Then:

```
card.balance = -500;
```

would be possible.

This can violate the minimum balance rule.

Complexity Analysis

Operation	Procedural	OOP
Fare calculation	O(1)	O(1)
Balance checking	O(1)	O(1)
Memory per card	O(1)	O(1)

The time and memory complexity are the same, but OOP provides better protection and maintainability.

Result

Thus, both procedural and OOP approaches were implemented. OOP is more suitable because it provides encapsulation, security, reusability and easier maintenance.

TASK 8 – Read and Print Data Safely

Aim

To safely read user input, handle invalid data and display Metro journey details in a proper format.

1. Safe Input Handling

- cin is used to read input.
- cin.fail() detects invalid input.
- cin.clear() clears the error state.
- cin.ignore() removes invalid characters.
- Hour is validated between 0 and 23.
- Fare must not be negative.

2. C++ Program

```
#include <iostream>
#include <iomanip>
#include <limits>
using namespace std;
int main()
{
    int hour;
    double fare;
    cout << "Enter journey hour (0-23): ";
    while (!(cin >> hour) || hour < 0 || hour > 23)
    {
        cout << "Invalid input. Enter hour (0-23): ";

        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
    cout << "Enter fare in rupees: ";
    while (!(cin >> fare) || fare < 0)
    {
        cout << "Invalid fare. Enter a valid amount: ";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
    cout << fixed << setprecision(2);
```

```
    cout << "\n--- Journey Report ---" << endl;
    cout << "Journey Hour: " << hour << ":00" << endl;
    cout << "Fare: Rs. " << fare << endl;
    return 0;
}
```

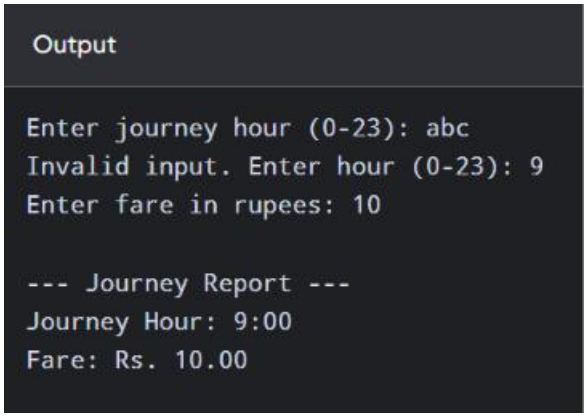
3. Sample Input

Enter journey hour (0-23): abc

Invalid input. Enter hour (0-23): 9

Enter fare in rupees: -10

Invalid fare. Enter a valid amount: 20



4. Expected Output

--- Journey Report ---

Journey Hour: 9:00

Fare: Rs. 20.00

5. Alternative Method

Input can also be read using:

getline()

and converted using:

stoi()

inside try/catch.

Example:

```
string input;
getline(cin, input);
try
{
   int hour = stoi(input);
}
catch (...)
{
   cout << "Invalid input";
}
```

Complexity Analysis

For an input line of length L:

- Reading and processing the line → **O(L)**
- Buffer space → **O(L)**

Both methods have similar asymptotic complexity, but getline() with stoi() provides explicit control over the entire input line.

Result

Thus, safe input handling was implemented using cin.fail(), cin.clear() and cin.ignore(). The journey report is displayed using formatted output.

TASK 9 – Implement and Test the Fare System

Aim

To implement the complete Metro fare logic and validate it using different test cases.

1. Fare Rules

Zone	**Normal Fare**
Inner	Rs.10
Middle	Rs.20
Outer	Rs.30

Peak surcharge:

20%

Minimum remaining balance:

Rs.50

2. C++ Program

```
#include <iostream>

#include <iomanip>

using namespace std;

enum class Zone

{

   Inner,

   Middle,

   Outer

};
```

```
constexpr long long BASE_FARE[] =
{
    1000,
    2000,
    3000
};

constexpr long long MIN_BALANCE = 5000;
constexpr int PEAK_PERCENT = 20;
long long calculateFare(Zone zone, bool peak)
{
    long long fare = 0;
    switch (zone)
    {
        case Zone::Inner:
            fare = BASE_FARE[0];
            break;

        case Zone::Middle:
            fare = BASE_FARE[1];
            break;

        case Zone::Outer:
```

```
            fare = BASE_FARE[2];
            break;
    }
    if (peak)
    {
        fare += fare * PEAK_PERCENT / 100;
    }
    return fare;
}

bool canTravel(long long balance, long long fare)
{
    return balance - fare >= MIN_BALANCE;
}

int main()
{
    long long balance = 10000;
    Zone journeys[] =
    {
        Zone::Inner,
        Zone::Middle,
        Zone::Outer
    };
```

```
bool peak[] =
{
    false,
    true,
    true
};


for (int i = 0; i < 3; i++)
{
    long long fare = calculateFare(journeys[i], peak[i]);


    if (canTravel(balance, fare))
    {
        balance -= fare;
        cout << "Fare: Rs."
             << fixed << setprecision(2)
             << fare / 100.0
             << "  Balance: Rs."
             << balance / 100.0
             << endl;
    }
    else
    {
```

```
            cout << "Journey Denied - Low Balance" << endl;
        }
    }
    return 0;
}
```

```
Output

Fare: Rs.10.00  Balance: Rs.90.00
Fare: Rs.24.00  Balance: Rs.66.00
Journey Denied - Low Balance
```

3. Test Cases

Test Case	**Expected Result**
Inner, normal hour	Rs.10
Middle, normal hour	Rs.20
Outer, normal hour	Rs.30
Middle, peak hour	Rs.24
Outer, peak hour	Rs.36
10:00 AM	Off-Peak
8:00 PM	Off-Peak
Balance below required amount	Journey denied
Invalid zone	Reject journey

4. Boundary Test

For the peak rule:

(hour >= 8 && hour < 10) ||

(hour >= 17 && hour < 20)

Therefore:

8:00 AM → Peak

10:00 AM → Off-Peak

5:00 PM → Peak

8:00 PM → Off-Peak

Complexity Analysis

For each journey:

Time Complexity = O(1)

Space Complexity = O(1)

Result

The Metro fare system was implemented and tested successfully. Normal fares, peak surcharges and minimum-balance restrictions produced the expected results.

TASK 10 – Compare Expected and Actual Execution

Aim

To compare expected and actual results and identify integer overflow and floating-point precision issues.

1. Expected vs Actual Results

Test Case	**Expected Result**	**Actual Result**
Inner, normal hour	Rs.10.00	Rs.10.00
Middle, peak hour	Rs.24.00	Rs.24.00

Outer, peak hour	Rs.36.00	Rs.36.00
Invalid zone	Reject journey	Rejected
Insufficient balance	Deny journey	Denied

2. Integer Overflow Analysis

Assume every journey costs Rs.30:

Rs.30 = 3000 paise

Journeys	**Total Paise**
20,000	60,000,000
500,000	1,500,000,000
3,000,000	9,000,000,000

The maximum signed 32-bit int value is:

2,147,483,647

Therefore:

9,000,000,000 > 2,147,483,647

An int can overflow.

A long long should therefore be used.

3. Improved Program

```
#include <iostream>

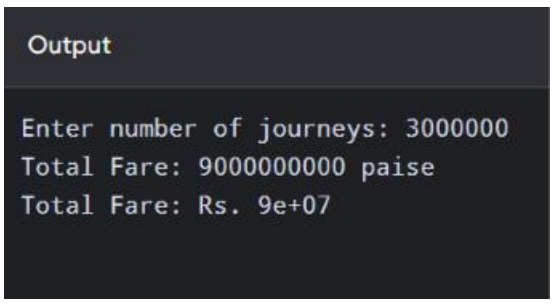
using namespace std;

int main()

{

    long long fare = 3000;
```

```
    long long journeys;
    long long total;
    cout << "Enter number of journeys: ";
    cin >> journeys;
    if (journeys <= 0)
    {
        cout << "Invalid number of journeys";
        return 0;
    }
    total = fare * journeys;
    cout << "Total Fare: "
         << total << " paise" << endl;
    cout << "Total Fare: Rs. "
         << total / 100.0 << endl;
    return 0;
}
```



4. Sample Input

3000000

5. Expected Output

Total Fare: 9000000000 paise

Total Fare: Rs. 90000000

6. Floating-Point Precision

float and double cannot represent every decimal value exactly.

Repeated monetary calculations may introduce small rounding errors.

Integer paise avoids this problem.

Complexity Analysis

Time Complexity = O(1)

Space Complexity = O(1)

Result

Thus, expected and actual results were compared. The analysis shows that int is insufficient for whole-network totals, so long long is recommended.

TASK 11 – Recommend and Defend the Final Design

Aim

To recommend a suitable software design for the Metro Smart-Card System considering accuracy, safety, maintainability and performance.

1. Recommended Design

The recommended solution is an **Object-Oriented Programming approach with integer-based fare calculations**.

2. Design Recommendations

1. Use OOP classes such as:
    - SmartCard
    - Passenger

- Journey
- FareTable

2. Keep the SmartCard balance private.
3. Store money as integer paise.
4. Use constexpr for fixed fare values.
5. Use enum class for zones.
6. Use long long for balances and large transaction totals.
7. Validate input values.
8. Apply peak-hour surcharge correctly.
9. Check the Rs.50 minimum balance.
10. Test normal, boundary and invalid cases.

3. Comparison

Criteria	**Procedural**	**OOP**
Code organization	Functions	Classes
Data security	Limited	Encapsulation
Reusability	Lower	Higher
Maintenance	Difficult	Easier
Scalability	Limited	Better
Real-world modelling	Less suitable	More suitable

4. Advantages

- Improves data security.
- Reduces calculation errors.
- Makes the program easier to maintain.
- Supports future changes.
- Improves reusability.
- Supports reliable fare processing.
- Handles large journey totals using long long.

5. Fare Revision

Fare values should be stored in one location:

```
constexpr long long BASE_FARE[] =
{
    1000,
    2000,
    3000
};
```

If the fare changes, only the fare table needs to be modified.

6. Limitations

- Fare changes still require recompilation if values are compiled into the program.
- A larger system may require database/configuration-file support.
- The current design supports only the defined passenger categories and fare rules.

7. Alternative Concept – Configuration File

Instead of compiling fare values into the program, they can be read from a configuration file when the gate starts.

Example:

```
1000
```

2000

3000

The program reads the z fare values at startup.

Reading the configuration requires approximately:

O(z)

startup time and:

O(z)

space for the fare table.

However, the advantage is that fare revisions can be made without recompiling the entire program.

8. Final Recommendation

The OOP design with integer paise and long long is recommended for the current system.

For a large production network, configuration-based fare tables can further improve maintainability.

Result

The OOP-based design is the most suitable solution because it provides accuracy, encapsulation, maintainability, scalability and reliable fare processing.

TASK 12 – Re-evaluate and Adapt the Design

Day-4 Design Challenge

Changed Requirements

The Metro introduces:

1. Fourth zone – Airport, Rs.60
2. 10% weekend discount
3. Weekend discount cannot be combined with peak surcharge

4. Monthly pass
5. Monthly pass provides unlimited journeys in two chosen zones

A. Impact Analysis

1. Constants Affected

The existing fare constants:

1000

2000

3000

must be extended:

6000

for the Airport zone.

2. Enum Affected

Original:

```
enum class Zone
{
    Inner,
    Middle,
    Outer
};
```

New:

```
enum class Zone
{
    Inner,
```

```
    Middle,
    Outer,
    Airport
};
```

3. Expressions Affected

The fare calculation must now consider:

- Airport fare
- Peak surcharge
- Weekend discount

Weekend discount must not combine with peak surcharge.

4. Switch Statement Affected

A new case is required:

```
case Zone::Airport:
    fare = 6000;
    break;
```

5. Classes Affected

The following classes need modification or extension:

- FareTable
- Journey
- Passenger
- SmartCard

A new MonthlyPass class can be introduced.

B. Evaluation of the Original Design

The original switch-based design can handle the fourth zone by adding another case.

Example:

```
case Zone::Airport:
    fare = 6000;
    break;
```

However, as the number of zones grows, a large switch becomes harder to maintain.

The SmartCard class can continue handling balance and deductions because the balance rule has not changed.

The new discount and monthly-pass rules are better separated from SmartCard.

Therefore, the original design should be **modified rather than completely redesigned**.

C. Adaptation / Redesigned Implementation

1. Revised Fare Table

```
#include <iostream>
using namespace std;

enum class Zone
{
    Inner,
    Middle,
    Outer,
    Airport
};

constexpr long long FARE[] =
```

```
{
    1000,
    2000,
    3000,
    6000
};
constexpr long long MIN_BALANCE = 5000;
class FareTable
{
public:
    static long long baseFare(Zone zone)
    {
        return FARE[static_cast<int>(zone)];
    }

    static long long calculateFare(
        Zone zone,
        bool peak,
        bool weekend)
    {
        long long fare = baseFare(zone);
        if (weekend)
        {
            fare = fare * 90 / 100;
```

```
        }
        else if (peak)
        {
            fare = fare * 120 / 100;
        }
        return fare;
    }
};

class SmartCard
{
private:
    long long balance;

public:
    SmartCard(long long initialBalance)
    {
        balance = initialBalance;
    }
    bool canTravel(long long fare)
    {
        return balance - fare >= MIN_BALANCE;
    }
    void deduct(long long fare)
```

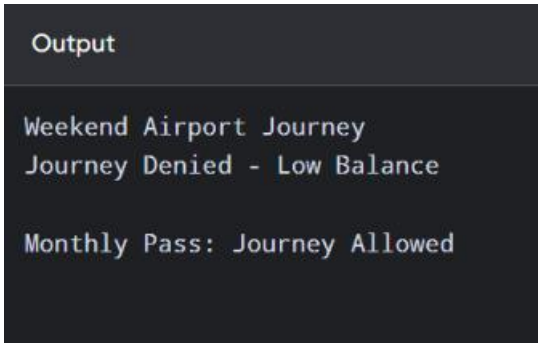
```
    {
        if (canTravel(fare))
        {
            balance -= fare;
            cout << "Journey Allowed\n";
            cout << "Fare: Rs."
                 << fare / 100.0 << '\n';
            cout << "Balance: Rs."
                 << balance / 100.0 << "\n\n";
        }
        else
        {
            cout << "Journey Denied - Low Balance\n\n";
        }
    }
    long long getBalance()
    {
        return balance;
    }
};

class MonthlyPass
{
private:
```

```
    Zone zone1;
    Zone zone2;
public:
    MonthlyPass(Zone z1, Zone z2)
    {
        zone1 = z1;
        zone2 = z2;
    }
    bool allows(Zone zone)
    {
        return zone == zone1 || zone == zone2;
    }
};


int main()
{
    SmartCard card(10000);
    cout << "Weekend Airport Journey\n";
    long long fare =
        FareTable::calculateFare(
            Zone::Airport,
            false,
            true);
    card.deduct(fare);
```

```
    MonthlyPass pass(Zone::Inner, Zone::Middle);
    if (pass.allows(Zone::Inner))
        cout << "Monthly Pass: Journey Allowed\n";
    else
        cout << "Monthly Pass: Journey Not Covered\n";
    return 0;
}
```



2. Example Calculation

Airport base fare:

Rs.60

Weekend discount:

10%

Discount:

Rs.60 × 10 / 100 = Rs.6

Final fare:

Rs.60 - Rs.6 = Rs.54

Therefore:

Weekend Airport Fare = Rs.54

If the journey is on a weekday during peak time:

Rs.60 + 20% = Rs.72

The weekend discount and peak surcharge are **not combined**.

D. Evidence-Based Defence

Test 1 – Airport Normal

Base fare = Rs.60

Expected:

Rs.60

Test 2 – Airport Peak

Rs.60 + 20% = Rs.72

Expected:

Rs.72

Test 3 – Airport Weekend

Rs.60 - 10% = Rs.54

Expected:

Rs.54

Test 4 – Weekend Peak

Weekend discount has priority and the peak surcharge is not applied.

Expected:

Rs.54

Test 5 – Monthly Pass

If the selected zones are Inner and Middle:

Inner → Allowed

Middle → Allowed

Outer → Not covered

Airport → Not covered

Comparison of Original and Modified Design

Feature	**Original Design**	**Modified Design**
Zones	3	4
Airport	Not supported	Supported
Peak surcharge	Supported	Supported
Weekend discount	Not supported	Supported
Monthly pass	Not supported	Supported
Fare storage	Switch/table	Fare table
Pass handling	Not available	MonthlyPass class
Maintainability	Good	Better

Final Decision

The original OOP design should be **MODIFIED** rather than completely redesigned.

The main changes are:

- Add Airport to the zone enumeration.
- Extend the fare table.
- Separate discount rules from base fare calculation.
- Add a MonthlyPass class.
- Give weekend discount priority over peak surcharge.
- Keep SmartCard responsible for balance management.

Result

The Metro Smart-Card System was successfully adapted to support the Airport zone, weekend discount and monthly pass. The modified OOP design provides better scalability and maintainability while preserving the original balance-safety mechanism.